

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 11: Network Flow II/III: Bipartite Matching and Applications

Foreign Trade University

Contents

1	Introduction: Flow as a Modeling Language	1
1.1	Roadmap	1
2	Bipartite Matching via Max Flow	2
2.1	Problem statement	2
2.2	The flow reduction	2
3	Hall’s Theorem and König’s Theorem	3
3.1	Hall’s marriage theorem	3
3.2	König’s theorem: matching = vertex cover	3
4	Disjoint Paths	4
4.1	Edge-disjoint paths and Menger’s theorem	4
4.2	Vertex-disjoint paths via node splitting	5
5	Circulations with Demands and Lower Bounds	5
6	Flow Applications I: Bipartite Assignment and Survey Design	6
7	Flow Applications II: Baseball Elimination	6
8	Flow Applications III: Project Selection (Max-Weight Closure)	7
9	Flow Applications IV: Image Segmentation as Min-Cut	7
10	Summary and Looking Ahead	8

1 Introduction: Flow as a Modeling Language

Last week we developed the theory of **maximum flow** and the **max-flow min-cut theorem**: in any flow network the value of a maximum s – t flow equals the capacity of a minimum s – t cut, and the Ford–Fulkerson family of algorithms (in particular Edmonds–Karp, which always augments along a shortest path) computes both. That theorem is powerful in its own right, but its real importance is as a *modeling language*: a surprising number of combinatorial problems that have nothing to do with “pipes and fluid” can be *reduced* to a max-flow or a min-cut computation. This week (Kleinberg & Tardos, Ch. 7) is about those reductions.

The recurring pattern is always the same:

- (1) Start with a combinatorial problem (a matching, a set of disjoint paths, a selection of projects, an image to segment).
- (2) **Build** a flow network whose structure encodes the constraints of the problem.
- (3) **Prove** a correspondence: feasible/optimal solutions of the original problem correspond exactly to flows/cuts of a certain value.
- (4) **Solve** the flow problem and **translate** the answer back.

The art is entirely in steps (2) and (3); step (4) is a black-box call to last week’s algorithm. Throughout, we use the shared `MaxFlow` helper (an Edmonds–Karp implementation in `starter_code.py`) as that black box, so the practical problems can focus on the *reductions*.

1.1 Roadmap

We begin with the cleanest and most important application – **bipartite matching** – and the two beautiful min-max theorems that surround it (**Hall’s theorem** and **König’s theorem**). We then treat **disjoint paths** (Menger’s theorem), **circulations with demands and lower bounds** (a more expressive flow model), and finally a sequence of “killer app” reductions: **bipartite assignment** / **survey design**, **baseball elimination**, **project selection** (max-weight closure via min-cut), and **image segmentation** (also a min-cut). Practical Problems 1–12 implement essentially all of these.

2 Bipartite Matching via Max Flow

2.1 Problem statement

Definition 2.1 (Bipartite graph). *A graph $G = (V, E)$ is bipartite if V can be partitioned into two sets X and Y such that every edge has one endpoint in X and one in Y . Equivalently (and this is what Problem 1 checks), G is bipartite iff it has no odd-length cycle, iff its vertices can be 2-colored so adjacent vertices differ.*

Definition 2.2 (Matching). *A matching $M \subseteq E$ is a set of edges no two of which share an endpoint. A vertex is matched (or saturated) if it is an endpoint of some edge of M , and free otherwise. A **maximum matching** is a matching of largest possible size. A perfect matching saturates every vertex.*

The **Maximum Bipartite Matching Problem** asks for a maximum matching in a given bipartite graph $G = (X \cup Y, E)$. This models any “pairing” task: assigning workers to jobs, students to projects, applicants to positions, etc.

2.2 The flow reduction

The capacity-1 edges out of s and into t enforce that each x is matched at most once and each y is matched at most once. Problem 3 implements exactly this reduction; Problem 2 implements the direct combinatorial alternative (Kuhn’s augmenting-path algorithm), and Problem 3’s tests *verify the two give equal matching size on random graphs*.

Theorem 2.3 (Matching = integer flow). *The maximum matching size in G equals the value of a maximum s – t flow in the network above. Moreover, every integer flow corresponds to a matching, and vice versa.*

Algorithm 1 Bipartite Matching via Max Flow

- 1: Create a source s and sink t .
 - 2: For each $x \in X$: add edge $s \rightarrow x$ with capacity 1.
 - 3: For each $y \in Y$: add edge $y \rightarrow t$ with capacity 1.
 - 4: For each edge $(x, y) \in E$: add edge $x \rightarrow y$ with capacity 1.
 - 5: Compute a maximum integer s - t flow f .
 - 6: **return** the set of edges $(x, y) \in E$ carrying one unit of flow.
-

Proof. Matching $\Rightarrow$ flow. Given a matching M of size k , route one unit of flow along $s \rightarrow x \rightarrow y \rightarrow t$ for each $(x, y) \in M$. Since M is a matching, no two of these paths share an x or a y , so every capacity-1 edge carries at most one unit; this is a feasible flow of value k .

Flow $\Rightarrow$ matching. Because all capacities are integers, by the integrality theorem (last week) there is an integer maximum flow f , in which every edge carries 0 or 1 unit. The middle edges $x \rightarrow y$ carrying one unit form a set M . Each x has at most one unit entering it (from s) and hence at most one unit leaving it, so x is in at most one edge of M ; likewise each y . Thus M is a matching, of size equal to the value of f .

The two directions show the maximum matching size equals the maximum flow value. $\square$

The reduction runs in $O(V \cdot E)$ time with Edmonds–Karp (each augmenting path increases the matching by one, and there are at most $\min(|X|, |Y|)$ of them), matching the running time of Kuhn’s algorithm. Problem 6 reuses this machinery to answer an *assignment feasibility* question: given people, tasks, and an “allowed” relation, can *everyone* be assigned a distinct task? That is exactly asking whether a matching saturating all people exists.

3 Hall’s Theorem and König’s Theorem

Two classical min-max theorems characterize bipartite matchings precisely. Both can be derived from max-flow min-cut, and both are implemented this week (Problems 4 and 5).

3.1 Hall’s marriage theorem

Definition 3.1 (Neighborhood and deficiency). *For a set $S \subseteq X$, its neighborhood $N(S) \subseteq Y$ is the set of all vertices adjacent to some vertex of S . We call S deficient if $|N(S)| < |S|$.*

Theorem 3.2 (Hall, 1935). *A bipartite graph $G = (X \cup Y, E)$ has a matching saturating every vertex of X (a perfect matching of X) **if and only if** $|N(S)| \geq |S|$ for every subset $S \subseteq X$.*

Proof. ($\Rightarrow$) If a matching saturates X , then for any $S \subseteq X$ the matched partners of the vertices of S are $|S|$ distinct vertices, all lying in $N(S)$; hence $|N(S)| \geq |S|$.

($\Leftarrow$) We prove the contrapositive using max-flow min-cut. Build the flow network of Theorem 2.3. The matching saturates all of X iff the max flow value equals $|X|$, iff (max-flow min-cut) the minimum cut has capacity $|X|$. Consider any s - t cut (A, B) with $s \in A$. Let $S = X \cap B$ (the X -vertices on the sink side) and note $X \cap A$ contributes capacity $|X \cap A|$ via the cut edges $s \rightarrow x$. To keep the cut finite (the middle edges have capacity 1 but we may instead cut the $y \rightarrow t$ edges), an optimal finite cut places $N(S) \subseteq A$ on the source side – otherwise an infinite-capacity... more carefully: the cheapest cut that “gives up” on the vertices of S must pay 1 for each $s \rightarrow x$ with $x \in X \cap A = X \setminus S$, namely $|X| - |S|$, plus 1 for each $y \rightarrow t$ with $y \in N(S)$ on the source side, namely $|N(S)|$. So the minimum cut capacity is $\min_{S \subseteq X} (|X| - |S| + |N(S)|)$.

This is less than $|X|$ exactly when some S has $|N(S)| < |S|$, i.e. some deficient set exists. Hence: no X -saturating matching $\iff$ a deficient set exists. Contrapositive: if no set is deficient, an X -saturating matching exists. $\square$

Problem 4 turns this proof into code: it decides whether X has a perfect matching and, when not, *returns an explicit deficient set S* (recovered from the alternating-path structure of a maximum matching). The deficient set is a *certificate* of infeasibility – a short, checkable reason why no perfect matching can exist.

3.2 König's theorem: matching = vertex cover

Definition 3.3 (Vertex cover). A vertex cover of $G = (V, E)$ is a set $C \subseteq V$ such that every edge has at least one endpoint in C . The **minimum vertex cover** is a smallest such set.

For any graph, a matching and a vertex cover are in tension:

Lemma 3.4 (Weak duality). In any graph, $|M| \leq |C|$ for every matching M and every vertex cover C . In particular, $\text{max-matching} \leq \text{min-vertex-cover}$.

Proof. The edges of M are pairwise disjoint, so they have $2|M|$ distinct endpoints. Every edge of M must be covered, i.e. have an endpoint in C ; distinct edges of M need distinct cover vertices (they share no endpoint). Hence $|C| \geq |M|$. $\square$

For *general* graphs this inequality can be strict: a triangle has maximum matching 1 but minimum vertex cover 2 (Problem 12 exhibits exactly this). In *bipartite* graphs, equality always holds:

Theorem 3.5 (König, 1931). In a bipartite graph, the size of a maximum matching equals the size of a minimum vertex cover.

Proof. By Lemma 3.4, $\text{max-matching} \leq \text{min-cover}$; we exhibit a vertex cover of size exactly $|M|$, where M is a maximum matching. Let $U \subseteq X$ be the set of X -vertices left *unmatched* by M . Mark every vertex reachable from U by an *alternating path* (a path that alternates non-matching edges $X \rightarrow Y$ and matching edges $Y \rightarrow X$). Let Z be the set of marked vertices, and define

$$C = (X \setminus Z) \cup (Y \cap Z).$$

C is a cover. Suppose some edge (x, y) has neither endpoint in C , i.e. $x \in X \cap Z$ (marked) and $y \in Y \setminus Z$ (unmarked). If $(x, y) \notin M$, then from the marked x we could extend an alternating path along the non-matching edge (x, y) to reach y , marking it – contradiction. If $(x, y) \in M$, then x was marked only by arriving along the matching edge (x, y) from y , so y would be marked – contradiction. Hence every edge is covered.

$|C| = |M|$. Every vertex of C is saturated by M : a vertex in $X \setminus Z$ is matched (the unmatched X -vertices are all in $U \subseteq Z$), and a vertex in $Y \cap Z$ is matched (an unmatched Y -vertex would complete an augmenting path from U , contradicting maximality of M). Moreover no matching edge has *both* endpoints in C : if $(x, y) \in M$ had $x \in X \setminus Z$ and $y \in Y \cap Z$, then y marked implies (since the only way to mark a matched y is via its matching partner) that x is marked, i.e. $x \in Z$, contradiction. So each edge of M contributes exactly one endpoint to C , and every vertex of C is the endpoint of exactly one matching edge – giving a bijection between M and C . Thus $|C| = |M|$. $\square$

This proof is *constructive*, and Problem 5 implements it directly: given a maximum matching (from Problem 2), it builds a vertex cover of size $|M|$ via the alternating-path marking, and the tests verify $|\text{cover}| = |\text{matching}|$ on random bipartite graphs. Problem 12 ties everything together by checking $\text{max-matching} \leq \text{min-cover}$ for general graphs and equality for bipartite graphs via brute force.

Corollary 3.6 (Hall from König). *Hall's theorem follows from König's: X has a perfect matching iff $\text{min-cover} = |X|$, and one checks that a cover smaller than $|X|$ exists iff some $S \subseteq X$ is deficient.*

4 Disjoint Paths

4.1 Edge-disjoint paths and Menger's theorem

Definition 4.1 (Edge-disjoint paths). *A collection of s - t paths is edge-disjoint if no edge is used by more than one path.*

Theorem 4.2 (Menger, edge version). *The maximum number of edge-disjoint s - t paths in a directed graph equals the minimum number of edges whose removal disconnects t from s (a minimum s - t edge cut).*

Proof. Assign every edge capacity 1 and compute a max flow. An integer flow of value k decomposes (by repeatedly tracing s - t paths along positive-flow edges, and discarding any cycles) into k edge-disjoint s - t paths, since each capacity-1 edge can belong to at most one path. Conversely k edge-disjoint paths give a flow of value k . So $\text{max edge-disjoint paths} = \text{max flow value} = \text{min cut capacity} = \text{max-flow min-cut} = \text{min number of edges to delete}$. $\square$

Problem 7 implements this: build the unit-capacity network, return the max flow.

4.2 Vertex-disjoint paths via node splitting

To forbid two paths from sharing an *internal vertex*, we use the **node-splitting** gadget:

Algorithm 2 Vertex-Disjoint Paths via Node Splitting

- 1: **for** each internal vertex v (i.e. $v \neq s, t$) **do**
 - 2: Split v into v_{in} and v_{out} , with an edge $v_{\text{in}} \rightarrow v_{\text{out}}$ of capacity 1.
 - 3: **end for**
 - 4: Replace each original edge (u, v) by $u_{\text{out}} \rightarrow v_{\text{in}}$ of capacity 1.
 - 5: Compute a max s - t flow; its value is the number of vertex-disjoint paths.
-

The internal capacity-1 edge $v_{\text{in}} \rightarrow v_{\text{out}}$ guarantees at most one unit of flow – hence at most one path – passes *through* v . Problem 8 implements this and its tests contrast it with Problem 7: two paths that are edge-disjoint but share a middle vertex are counted by Problem 7 but not by Problem 8.

5 Circulations with Demands and Lower Bounds

So far every flow had a single source and sink. Many problems instead have *multiple* supplies and demands and require *minimum* as well as maximum amounts on edges. The **circulation** model captures this.

Definition 5.1 (Circulation with demands). *Each vertex v has a demand d_v : if $d_v > 0$, v must absorb d_v units (a sink); if $d_v < 0$, v supplies $-d_v$ units (a source). Each directed edge e has a capacity c_e and possibly a lower bound $\ell_e \geq 0$. A feasible circulation assigns a flow $f_e \in [\ell_e, c_e]$ to every edge so that for each vertex v ,*

$$\sum_{e \text{ into } v} f_e - \sum_{e \text{ out of } v} f_e = d_v.$$

A necessary condition is $\sum_v d_v = 0$ (total supply equals total demand).

Theorem 5.2 (Feasibility reduces to max flow). *A feasible circulation with demands and lower bounds can be decided by a single max-flow computation, as follows.*

Construction. Step 1 – eliminate lower bounds. For each edge (u, v) with bounds $[\ell, c]$, “pre-push” ℓ units: set its capacity to $c - \ell$ and update demands $d_u += \ell$ and $d_v -= \ell$. (We have committed ℓ units to leave u and arrive at v ; the residual problem has lower bound 0.)

Step 2 – super source and sink. Add S and T . For each vertex v with adjusted demand $d_v < 0$ (net supply), add $S \rightarrow v$ with capacity $-d_v$. For each v with $d_v > 0$ (net demand), add $v \rightarrow T$ with capacity d_v .

Step 3 – solve. A feasible circulation exists *iff* the maximum S – T flow *saturates every edge out of S* – equivalently, equals $D := \sum_{v: d_v > 0} d_v$, the total adjusted demand (and this requires the adjusted supply and demand to balance). Saturating all source edges means every supply is fully shipped and every demand fully met, which is exactly a feasible circulation; conversely a feasible circulation yields such a saturating flow. $\square$

Problem 11 implements this feasibility test, including the lower-bound elimination and the balance check ($\sum d_v = 0$ after adjustment). This model is the foundation for many scheduling and routing feasibility questions, and it generalizes the survey-design and assignment problems of the next section.

6 Flow Applications I: Bipartite Assignment and Survey Design

The bipartite-matching reduction generalizes smoothly when each vertex can be used several times. In **survey design**, a magazine wants each *customer* i to be asked between c_i and c'_i questions, and each *product* j to receive between p_j and p'_j reviews, where customer i may only be asked about products they have purchased. Model it as a circulation / bounded flow:

- $s \rightarrow \text{customer } i$ with bounds $[c_i, c'_i]$;
- $\text{customer } i \rightarrow \text{product } j$ with bounds $[0, 1]$ if i purchased j ;
- $\text{product } j \rightarrow t$ with bounds $[p_j, p'_j]$;
- a return edge $t \rightarrow s$ of unbounded capacity (turning the design into a circulation).

A feasible integer flow is exactly a valid survey design. This is the “bounded-degree” generalization of bipartite matching (Problem 6’s assignment question is the special case where every bound is $[0, 1]$ and we ask the flow to saturate all customers).

7 Flow Applications II: Baseball Elimination

A delightful, non-obvious use of max flow decides whether a sports team has been mathematically *eliminated* from finishing first.

Definition 7.1 (Baseball elimination). *We are given teams with current win totals w_i and a schedule of g_{ij} games remaining between teams i and j . Team x is eliminated if there is no way to assign winners to all remaining games so that x finishes with at least as many wins as every other team.*

Let $W = w_x + (\text{remaining games of } x)$ be the most wins x can finish with. If some team already has $w_i > W$, x is trivially eliminated. Otherwise build a flow network (Kleinberg & Tardos, §7.12):

Algorithm 3 Baseball Elimination as Max Flow

- 1: **for** each unordered pair $\{i, j\}$ of *other* teams with $g_{ij} > 0$ **do**
 - 2: Add a *game node* $\{i, j\}$ with edge $s \rightarrow \{i, j\}$ of capacity g_{ij} .
 - 3: Add edges $\{i, j\} \rightarrow i$ and $\{i, j\} \rightarrow j$ of capacity ∞ .
 - 4: **end for**
 - 5: **for** each other team i **do**
 - 6: Add edge $i \rightarrow t$ of capacity $W - w_i$ ▷ i 's remaining “win budget”
 - 7: **end for**
 - 8: Compute the max s – t flow.
 - 9: **return** “not eliminated” iff the flow saturates *all* game edges.
-

Each game $\{i, j\}$ sends one unit per game played; that unit must flow to the winner (i or j), and the edge $i \rightarrow t$ caps how many additional wins team i may accumulate before passing W . If *all* remaining games can be routed (the flow saturates every $s \rightarrow \{i, j\}$ edge), then there is a way to play out the season with x tied for first or better; otherwise x is eliminated. Problem 9 implements this on a small dataset, including the trivial elimination shortcut.

The minimum cut, by the way, yields a *certificate of elimination*: a subset R of teams whose combined current and remaining intra-group wins force their average above W – a human-readable reason why x cannot catch up.

8 Flow Applications III: Project Selection (Max-Weight Closure)

Definition 8.1 (Project selection). *We have projects, each with a profit p_v (possibly negative – a cost). Some projects have prerequisites: an edge (p, q) means “selecting p requires also selecting q .” A set S of projects is feasible if it is closed under prerequisites (if $p \in S$ and (p, q) is a prerequisite, then $q \in S$). The goal is a feasible S maximizing $\sum_{v \in S} p_v$.*

This is the **maximum-weight closure** problem, and it is solved by a *minimum cut* (Kleinberg & Tardos, §7.11):

Theorem 8.2. *The maximum profit equals $P - C$, where C is the minimum cut, and the source side of the minimum cut is an optimal feasible project set.*

Sketch. The infinite-capacity prerequisite edges guarantee that any *finite* cut never separates a selected project from a prerequisite – so the source side S is always closed. For such a cut, the cut capacity equals $\sum_{v \in S, p_v < 0} (-p_v) + \sum_{v \notin S, p_v > 0} p_v$, i.e. the forgone positive profits plus the incurred costs. Minimizing this is the same as maximizing $\sum_{v \in S} p_v = P - (\text{cut})$. Hence min-cut $\Leftrightarrow$ max-profit closure. $\square$

Algorithm 4 Project Selection via Min-Cut

```
1: Let  $P = \sum_{v: p_v > 0} p_v$  (sum of positive profits).
2: for each project  $v$  with  $p_v > 0$  do add  $s \rightarrow v$  with capacity  $p_v$ .
3: end for
4: for each project  $v$  with  $p_v < 0$  do add  $v \rightarrow t$  with capacity  $-p_v$ .
5: end for
6: for each prerequisite  $(p, q)$  do add  $p \rightarrow q$  with capacity  $\infty$ .
7: end for
8: Compute the minimum  $s$ - $t$  cut, of value  $C$ .
9: return profit  $P - C$ , and  $S = \{v : v \text{ on the source side}\}$ .
```

Problem 10 implements both the min-cut solution and a brute-force search over all closed subsets, and the tests confirm they agree on random small instances. The source side of the cut is recovered with `MaxFlow.min_cut_reachable`.

9 Flow Applications IV: Image Segmentation as Min-Cut

A final, visually compelling application: **image segmentation**. We wish to label each pixel as *foreground* or *background*. Each pixel i has a *likelihood* $a_i \geq 0$ of being foreground and $b_i \geq 0$ of being background, and each pair of neighboring pixels (i, j) has a *separation penalty* $p_{ij} \geq 0$ we pay if we split them across the two labels (this enforces smooth, contiguous regions).

We *maximize* $\sum_{i \in F} a_i + \sum_{j \in B} b_j - \sum_{(i,j) \text{ split}} p_{ij}$, which is equivalent to *minimizing* $\sum_{i \in F} b_i + \sum_{j \in B} a_j + \sum_{(i,j) \text{ split}} p_{ij}$. Build a network with s = “foreground terminal,” t = “background terminal”:

- $s \rightarrow i$ of capacity a_i for every pixel i ;
- $i \rightarrow t$ of capacity b_i for every pixel i ;
- between neighbors, *two* directed edges $i \rightarrow j$ and $j \rightarrow i$, each of capacity p_{ij} .

A minimum s - t cut places foreground pixels on the source side and background on the sink side; its capacity is exactly the quantity to be minimized, with the neighbor edges contributing p_{ij} precisely when i and j end up on opposite sides. Thus image segmentation is *another* min-cut problem – the same machinery as project selection, with a different network. (The shared `MaxFlow.min_cut_reachable` helper would recover the foreground set.)

10 Summary and Looking Ahead

This week we treated max flow as a **modeling and reduction tool**, and saw how a single black-box algorithm (Edmonds–Karp, from Week 10) solves a wide range of combinatorial problems:

- **Bipartite matching** = unit-capacity max flow (Theorem 2.3); Problems 2–3, 6.
- **Hall’s theorem** (Theorem 3.2) and **Kőnig’s theorem** (Theorem 3.5): perfect matchings via the deficiency condition, and max-matching = min-vertex-cover in bipartite graphs; Problems 4, 5, 12.

- **Disjoint paths** (Menger, Theorem 4.2): edge-disjoint via unit capacities, vertex-disjoint via node splitting; Problems 7–8.
- **Circulations with demands and lower bounds** (Theorem 5.2): feasibility by one max-flow call; Problem 11.
- **Min-cut applications**: baseball elimination (Problem 9), project selection / max-weight closure (Theorem 8.2, Problem 10), and image segmentation – all decided by a minimum cut.

The unifying lesson is that the *hard work is the reduction*: encoding constraints as capacities, recognizing when a min-max theorem applies, and proving the correspondence between combinatorial objects and flows/cuts. Once that is done, optimality and a polynomial running time come for free from the max-flow min-cut theorem.

Looking ahead. Flow has given us a large catalog of problems that are *efficiently solvable*. Next week we turn to the other side of the coin: **intractability** and **NP-completeness** (Kleinberg & Tardos, Ch. 8). We will study *reductions* again – but now to argue that certain problems are *at least as hard* as a whole class of others, so that a polynomial algorithm for one would solve them all. Many problems that look deceptively close to the matchings and cuts of this week (e.g. maximum matching in *general* graphs is still polynomial, but maximum independent set, or 3-dimensional matching, are NP-complete) sit just beyond the reach of flow – and understanding exactly where that boundary lies is the subject of the rest of the course.